

Implementación de servicios RESTful (I)

El objetivo de esta práctica es aprender a implementar nuestro propio servicio RESTful empleando *Spring Boot*. Para ello, implementaremos operaciones básicas de lectura, creación, modificación y eliminación (GET, POST, PUT y DELETE) y comprobaremos el correcto funcionamiento de la API implementada usando *Postman*.

En futuras prácticas extenderemos la funcionalidad de esta API conectándola a una base de datos, añadiendo validación más avanzada, filtrado y paginación.

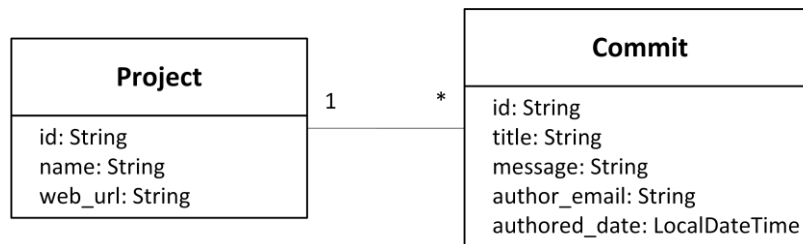


Figura 1: Modelo de datos

Las operaciones a implementar en esta práctica son las siguientes (en futuras prácticas añadiremos nuevas operaciones):

Recurso Project

HTTP	URI	Descripción
GET	/api/projects	Lista todos los proyectos de la aplicación.
GET	/api/projects/{projectId}	Devuelve el proyecto con id=projectId.
POST	/api/projects	Añade un nuevo proyecto cuyos datos se pasan en el cuerpo de la petición en formato JSON (no se debe pasar el id, se genera automáticamente). Si el proyecto se añade satisfactoriamente, devuelve "201 Created" y el contenido del proyecto, que por defecto contiene cero commits.
PUT	/api/projects/{projectId}	Actualiza el proyecto con id=projectId con los datos que se pasan en el cuerpo de la petición en formato JSON. Si el proyecto se actualiza correctamente, la API devuelve "204 No Content".
DELETE	/api/projects/{projectId}	Elimina el proyecto con id=projectId. Si el proyecto se elimina correctamente, la API devuelve "204 No Content".

Cada proyecto tiene un *identificador*, *nombre*, *url* y una *lista de commits*. En esta práctica, la lista de commits siempre estará vacía. En futuras prácticas veremos cómo conectar estos dos recursos. La representación JSON del recurso es:

```
{
  "id": "p1",
  "name": "Project name",
  "web_url": "www.project.com",
  "commits": []
}
```

Recurso Commit

HTTP	URI	Descripción
GET	/api/commits	Lista todos los commits de la aplicación.
GET	/api/commits/{commitId}	Devuelve el commit con id=commitId.
POST	/api/commits	Añade un nuevo commit cuyos datos se pasan en el cuerpo de la petición en formato JSON (no se debe pasar el id, se genera automáticamente). Si el commit se añade satisfactoriamente, devuelve "201 Created" y el contenido del commit.
PUT	/api/commits/{commitId}	Actualiza el commit con id=commitId con los datos que se pasan en el cuerpo de la petición en formato JSON. Si el commit se actualiza correctamente, la API devuelve "204 No Content".
DELETE	/api/commits/{commitId}	Elimina el commit con id=commitId. Si el commit se elimina correctamente, la API devuelve "204 No Content".

Cada **commit** tiene un *identificador*, *título*, *mensaje*, *email del autor* y *fecha*. La representación JSON del recurso es:

```
{
  "id": "c1",
  "title": "Commit title",
  "message": "Commit message",
  "author_email": "author@gmail.com",
  "authored_date": "2025-04-01T07:10:00"
}
```

Ejercicios

Ejercicio 1. Creación del modelo

- a) Cree un proyecto base de Spring Boot utilizando <https://start.spring.io/> y ábralo con IntelliJ. Los valores de configuración son los mismos que los usados en prácticas anteriores:
 - *Project*: Maven Project
 - *Language*: Java
 - *Spring Boot*: 3.5.12 (o similar)
 - *Packaging*: Jar
 - *Java*: 21
 - *Dependencies*: Spring Web, Spring Data JPA, H2 Database, Validation.
- b) Una vez importado el proyecto, cree un paquete “model” y añada una clase por cada recurso (Project y Commit), con sus correspondientes atributos según la Fig.1 y sus métodos getters y setters.

Ejercicio 2. Creación del repositorio con operaciones de lectura

- a) Cree un paquete “repository” y añada una clase por cada recurso. No olvide utilizar las anotaciones de Spring necesarias.
- b) Defina datos de prueba para ejecutar las operaciones (una lista de proyectos y de commits). Utilice la instrucción `UUID.randomUUID().toString()` para crear identificadores únicos aleatorios.
- c) Implemente, para cada recurso, un método que devuelva todos los objetos de un repositorio (e.g., `public List<Project> findAll()`) y uno que devuelva un solo objeto, filtrando por id (e.g., `public Project findOne(String id)`).

Ejercicio 3. Creación de controlador con operaciones de lectura

- a) Cree un paquete “controller” y añada una clase por cada recurso. No olvide utilizar las anotaciones de Spring necesarias.
- b) Implemente las operaciones de consulta (GET) de ambos recursos, empleando para ello los métodos que ha definido en los repositorios.
- c) Una vez implementadas las operaciones, cree una colección en *Postman* y compruebe su correcto funcionamiento. En concreto, implemente *tests* que realicen las siguientes comprobaciones:
 - Las operaciones de consulta devuelven un código 200.
 - Cada elemento devuelto por la API contiene las propiedades definidas en el modelo de datos. Por ejemplo, todo proyecto debe contener las propiedades “id”, “name”, “web_url” y “commits”.

Ejercicio 4. Implementación de operaciones de creación

Extienda el repositorio y el controlador para implementar operaciones de creación (POST) en ambos recursos. Compruebe el correcto funcionamiento de las operaciones utilizando *Postman* y defina pruebas que realicen las siguientes comprobaciones:

- La operación devuelve un código 201.
- Por defecto, la lista de commits de un proyecto debe estar vacía.
- Cada elemento devuelto por la API contiene las propiedades definidas en el modelo de datos.

Ejercicio 5. Implementación de operaciones de modificación

Extienda el repositorio y el controlador para implementar operaciones de modificación (PUT) en ambos recursos. Compruebe el correcto funcionamiento de las operaciones utilizando Postman y defina pruebas que realicen la siguiente comprobación:

- La operación devuelve un código 204.

Ejercicio 6. Implementación de operaciones de eliminación

Extienda el repositorio y el controlador para implementar operaciones de eliminación (DELETE) en ambos recursos. Compruebe el correcto funcionamiento de las operaciones utilizando Postman y defina pruebas que realicen la siguiente comprobación:

- La operación devuelve un código 204.